Abstract

This project focuses on simulating the behavior of several CPU scheduling algorithms to help students understand the practical implementation of these algorithms in operating systems. The simulator is implemented in Java and provides a user-friendly console interface where users can experiment with four core CPU scheduling algorithms: First-Come, First-Served (FCFS), Shortest Job First (SJF) (both Non-preemptive and Preemptive), and Round Robin (RR). Each algorithm is evaluated based on key performance metrics, such as waiting time and turnaround time.

The simulator allows users to input arrival times and burst times for different processes, with built-in input validation to ensure correct data entry, and then computes the scheduling results accordingly. In addition to the scheduling algorithms, the simulator generates visual outputs like process-level tables and Gantt charts for each algorithm, helping users visualize how processes are scheduled over time.

The primary goal of this project is to provide an educational tool that enables students to explore and compare the effectiveness of different scheduling algorithms in terms of their performance and efficiency. For each algorithm, the program prints detailed results, including **waiting time** and **turnaround time** averages, and offers a deeper understanding of how different algorithms behave in various scenarios.

Through this project, users can observe key concepts such as the **convoy effect** (in FCFS), **preemption** (in RR and SJF Preemptive), and how the system handles processes with

varying burst times. Additionally, the results generated help to identify the strengths and weaknesses of each algorithm, aiding in the learning process.

The code is designed to be simple and self-contained, allowing students to easily modify and extend it for further experimentation. The simulation serves as both an instructional tool and a practical resource for exploring the trade-offs between different scheduling strategies.

Table of contents

List of Figures

Introduction

This project focuses on simulating the behavior of different CPU scheduling algorithms. Understanding these algorithms is essential in the design of modern operating systems, as they significantly impact process execution time and system efficiency. In this report, we implement and test four scheduling algorithms:

1. First-Come, First-Served (FCFS): A simple scheduling algorithm where the first process to arrive is the first to be executed.
2. Round Robin (RR): A preemptive scheduling algorithm where each process is given a fixed time slice (quantum).
3. Shortest Job First (SJF): Both non-preemptive and preemptive versions of this algorithm schedule processes based on the shortest burst time, with the preemptive version allowing processes to be interrupted.

In addition to implementing these algorithms, the simulator incorporates robust error handling using try-catch blocks to validate user input. This ensures the program remains stable when users enter invalid data, such as letters instead of numbers or incorrect formats. The input validation enhances usability and prevents unexpected crashes during runtime.

CPU Scheduling Simulator

This section will describe the logic of the CPU scheduling simulator. The program simulates the behavior of the algorithms based on the arrival time and burst time input by the user. The results are then calculated, showing the waiting times and turnaround times for each process.

Error Handling and Input Validation

To ensure the reliability of the simulator, input validation is implemented using Java's try-catch blocks. This handles incorrect or unexpected inputs such as non-integer values or mismatched process counts. Instead of crashing, the program prompts users to re-enter valid data, enhancing usability and preventing runtime errors.

Sample Output:

```
run:
Enter number of processes:
h
Invalid input. Please enter a positive integer.
Enter number of processes:
i
Invalid input. Please enter a positive integer.
Enter number of processes:
j
Invalid input. Please enter a positive integer.
Enter number of processes:
e456
Invalid input. Please enter a positive integer.
Enter number of processes:
```

Figure 1. Error Handling and Input Validation Results

Expected Inputs

The CPU scheduling simulator requires specific user inputs to execute the selected algorithm accurately. The expected inputs are as follows:

- Number of Processes: A positive integer indicating the total number of processes to be scheduled.

- Arrival Time Specification: A confirmation (yes/no) from the user to determine whether individual arrival times will be provided.
    - If "yes," the program prompts for space-separated integer values representing the arrival times of all processes.
    - If "no," arrival times default to zero, implying that all processes arrive simultaneously at time zero.
- **Burst Times:**

  Space-separated integer values indicating the CPU burst duration for each process.
- **Quantum Value (Round Robin only):**

  When the Round Robin algorithm is selected, the simulator prompts for an additional integer input representing the quantum (time slice) to be allocated to each process.
- **Input Validation and Error Handling:**

  To ensure the program functions reliably, user inputs are validated using Java's try-catch mechanism. Invalid inputs (e.g., non-integer or negative values) are caught, and the user is prompted to re-enter valid data. This prevents runtime errors and ensures consistent behavior regardless of user mistakes.

Expected Outputs

Upon execution of the chosen scheduling algorithm, the simulator provides the following outputs:

1. **Process Scheduling Table:**

   A detailed tabular representation containing:

   - Process ID
   - Arrival Time
   - Burst Time
   - Waiting Time (in milliseconds)
   - Turnaround Time (in milliseconds)

2. **Performance Metrics:**

   - Average Waiting Time (in milliseconds)
   - Average Turnaround Time (in milliseconds)

3. **Gantt Chart:**

   A visual timeline illustrating the execution order of processes along with their start and end times.

Algorithm Implementations

1. FCFS Algorithm

Conceptual Explanation: FCFS is a non-preemptive algorithm where processes are executed in the order of their arrival. The process that arrives first gets executed first. There is no context switching, so it is easy to implement but it may not always provide the optimal performance.

Code Snippet:

```
static void fcfs(List<Process> processes) {
        processes.sort(Comparator.comparingInt(p -> p.arrivalTime));
        int time = 0;
        for (Process p : processes) {
            if (time < p.arrivalTime) time = p.arrivalTime;
            p.executionLog.add(new int[]{time, time + p.burstTime});
            p.waitingTime = time - p.arrivalTime;
            time += p.burstTime;
            p.turnaroundTime = p.waitingTime + p.burstTime;}
        printResult("FCFS", processes);}
```

Sample Output:

```
--- FCFS Algorithm ---

Process ID      Arrival Time    Burst Time      Waiting Time    Turnaround Time
1               0               10              0 ms            10 ms
2               3               4               7 ms            11 ms
3               6               5               8 ms            13 ms
4               9               2               10 ms           12 ms
Average Waiting Time: 6.25 ms
Average Turnaround Time: 11.50 ms

Gantt Chart:
|  P1  |  P2  |  P3  |  P4  |
0     10     14     19     21
```

Figure 2. CPU-scheduling results for the FCFS

2. Round Robin (RR) Algorithm

Conceptual Explanation: Round Robin (RR) is a preemptive scheduling algorithm where each process is given a fixed time slice (quantum). If the process does not finish within the time slice, it has put back in the ready queue.

Code Snippet:

```
static void roundRobin(List<Process> processes, int quantum) {
        Queue<Process> queue = new LinkedList<>();
        int time = 0, completed = 0;
        List<Process> arrived = new ArrayList<>(processes);
        arrived.sort(Comparator.comparingInt(p -> p.arrivalTime));
        Set<Integer> inQueue = new HashSet<>();

        while (completed < processes.size()) {
            for (Process p : arrived) {
                if (p.arrivalTime <= time && !inQueue.contains(p.id) &&
p.remainingTime > 0) {
                    queue.add(p);
                    inQueue.add(p.id);
                }
            }
            if (queue.isEmpty()) {
                time++;
                continue;
            }
            Process current = queue.poll();
            int executeTime = Math.min(quantum, current.remainingTime);
            if (current.startTime == -1) current.startTime = time;
            current.executionLog.add(new int[]{time, time + executeTime});
            time += executeTime;
            current.remainingTime -= executeTime;

            for (Process p : arrived) {
                if (p.arrivalTime <= time && !inQueue.contains(p.id) &&
p.remainingTime > 0) {
                    queue.add(p);
                    inQueue.add(p.id);
                }
            }
            if (current.remainingTime > 0) {
                queue.add(current);
            } else {
```

```
                current.turnaroundTime = time - current.arrivalTime;
                current.waitingTime = current.turnaroundTime -
current.burstTime;
                completed++;
            }
        }
        processes.sort(Comparator.comparingInt(p -> p.id));
        printResult("Round Robin", processes);
    }
```

Sample Output: (quantum = 3)

```
--- Round Robin Algorithm ---

Process ID      Arrival Time    Burst Time      Waiting Time    Turnaround Time
1               0               10              11 ms           21 ms
2               3               4               6 ms            10 ms
3               6               5               9 ms            14 ms
4               9               2               4 ms            6 ms
Average Waiting Time: 7.50 ms
Average Turnaround Time: 12.75 ms

Gantt Chart:
|  P1  |  P1  |  P1  |  P1  |  P2  |  P2  |  P3  |  P3  |  P4  |
0     3     6     9     12     13     15     18     20     21
```

Figure 3. CPU-scheduling results for the Round Robin

3. SJF (Non-preemptive) Algorithm

Conceptual Explanation: SJF is a non-preemptive algorithm that schedules the process with the shortest burst time that is ready to execute. Once a process starts, it runs to completion.

Code Snippet:

```
static void sjfNonPreemptive(List<Process> processes) {
```

```
        List<Process> result = new ArrayList<>();
        int time = 0;
        while (!processes.isEmpty()) {
            List<Process> available = new ArrayList<>();
            for (Process p : processes) {
                if (p.arrivalTime <= time) available.add(p);
            }
            if (available.isEmpty()) {
                time++;
                continue;
            }
            available.sort(Comparator.comparingInt(p -> p.burstTime));
            Process current = available.get(0);
            processes.remove(current);
            current.executionLog.add(new int[]{time, time +
current.burstTime});
            current.waitingTime = time - current.arrivalTime;
            time += current.burstTime;
            current.turnaroundTime = current.waitingTime + current.burstTime;
            result.add(current);
        }
        result.sort(Comparator.comparingInt(p -> p.id));
        printResult("SJF (Non-preemptive)", result);
    }
```

Sample Output:

```
--- SJF (Non-preemptive) Algorithm ---

Process ID      Arrival Time    Burst Time      Waiting Time    Turnaround Time
1               0               10              0 ms            10 ms
2               3               4               9 ms            13 ms
3               6               5               10 ms           15 ms
4               9               2               1 ms            3 ms
Average Waiting Time: 5.00 ms
Average Turnaround Time: 10.25 ms

Gantt Chart:
|  P1  |  P2  |  P3  |  P4  |
0     10     12     16     21
```

Figure 4. CPU-scheduling results for the SJF (Non-preemptive)

4. SJF (Preemptive) Algorithm

Conceptual Explanation: SJF Preemptive selects the process with the shortest remaining burst time at every step. If a shorter process arrives during execution, the current process is preempted.

Code Snippet:

```
static void sjfPreemptive(List<Process> processes) {
        int time = 0, completed = 0;
        Process current = null;
        while (completed < processes.size()) {
            Process shortest = null;
            for (Process p : processes) {
                if (p.arrivalTime <= time && p.remainingTime > 0) {
                    if (shortest == null || p.remainingTime <
shortest.remainingTime) {
                        shortest = p;
                    }
                }
            }
            if (shortest == null) {
                time++;
                continue;
            }
```

```
            if (shortest.startTime == -1) shortest.startTime = time;
            if (current != shortest) {
                current = shortest;
                current.executionLog.add(new int[]{time, time + 1});
            } else {
                int[] last =
current.executionLog.remove(current.executionLog.size() - 1);
                current.executionLog.add(new int[]{last[0], time + 1});
            }
            shortest.remainingTime--;
            time++;
            if (shortest.remainingTime == 0) {
                shortest.turnaroundTime = time - shortest.arrivalTime;
                shortest.waitingTime = shortest.turnaroundTime -
shortest.burstTime;
                completed++;
            }
        }
        processes.sort(Comparator.comparingInt(p -> p.id));
        printResult("SJF (Preemptive)", processes);
    }
```

Sample Output:

```
--- SJF (Preemptive) Algorithm ---

Process ID      Arrival Time    Burst Time      Waiting Time    Turnaround Time
1               0               10              11 ms           21 ms
2               3               4               0 ms            4 ms
3               6               5               3 ms            8 ms
4               9               2               0 ms            2 ms
Average Waiting Time: 3.50 ms
Average Turnaround Time: 8.75 ms

Gantt Chart:
|  P1  |  P1  |  P2  |  P3  |  P3  |  P4  |
0     3     7     9     11     14     21
```

Figure 5. CPU-scheduling results for the SJF (preemptive)

README (Compile, Run & Extend Guide)

Welcome!

This program simulates how operating systems schedule tasks using:

- First-Come First-Served (FCFS)
- Round Robin (RR)
- Shortest Job First (Non-preemptive)
- Shortest Job First (Preemptive / SRTF)

It's built in plain Java and ready to run in NetBeans 16 under the project name:

Mavenproject188.

What You'll Need

- NetBeans 16 (You already have it—great!)

- Java JDK 17 or newer installed
- (NetBeans 16 supports Java 17 by default. Just make sure your project is using the correct JDK. Or **visual studio code**)

How to Run the Simulator

A. Open NetBeans 16
B. Open the project: Mavenproject188
C. Paste the code into the main class (CPUSchedulingSimulator.java)
D. Right-click the project name and select Run
E. The simulator will launch in the Output window
F. Follow the prompts in the console to simulate CPU scheduling!
G. The simulator handles invalid inputs (like letters or missing values) gracefully, asking the user to try again instead of crashing.

Extend the Simulator

- You can add more algorithms such as Priority Scheduling
- The Process class can be expanded with fields like priority, I/O time, etc.
- The code is modular and beginner-friendly for customization

Note:

1. You only need to enter the quantum if you select Round Robin
2. Arrival times are optional—if not entered, they default to zero

Full Source :

```
import java.util.*;

class Process {
```

```
    int id, arrivalTime, burstTime, waitingTime, turnaroundTime, remainingTime,
startTime;
    List<int[]> executionLog = new ArrayList<>();

    Process(int id, int arrivalTime, int burstTime) {
        this.id = id;
        this.arrivalTime = arrivalTime;
        this.burstTime = burstTime;
        this.remainingTime = burstTime;
        this.startTime = -1;
    }
}

class BankerAlgorithm {
    int n, m;
    int[][] max;
    int[][] allocation;
    int[][] need;
    int[] available;
    int[] safeSequence;

    public BankerAlgorithm(int n, int m, int[][] max, int[][] allocation, int[]
available) {
        this.n = n;
        this.m = m;
        this.max = max;
        this.allocation = allocation;
        this.available = available.clone();
        this.need = new int[n][m];
        this.safeSequence = new int[n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                need[i][j] = max[i][j] - allocation[i][j];
            }
        }
    }

    public boolean isSafe() {
        int[] work = available.clone();
        boolean[] finish = new boolean[n];
        int count = 0;

        while (count < n) {
            boolean found = false;
```

```
            for (int i = 0; i < n; i++) {
                if (!finish[i]) {
                    boolean canAllocate = true;
                    for (int j = 0; j < m; j++) {
                        if (need[i][j] > work[j]) {
                            canAllocate = false;
                            break;
                        }
                    }

                    if (canAllocate) {
                        for (int j = 0; j < m; j++) {
                            work[j] += allocation[i][j];
                        }
                        safeSequence[count++] = i;
                        finish[i] = true;
                        found = true;
                    }
                }
            }
            if (!found) break;
        }

        return count == n;
    }

    public boolean requestResources(int processId, int[] request) {
        for (int j = 0; j < m; j++) {
            if (request[j] > need[processId][j]) {
                return false;
            }
        }

        for (int j = 0; j < m; j++) {
            if (request[j] > available[j]) {
                return false;
            }
        }

        for (int j = 0; j < m; j++) {
            available[j] -= request[j];
            allocation[processId][j] += request[j];
            need[processId][j] -= request[j];
        }
```

```
        if (isSafe()) {
            return true;
        } else {
            for (int j = 0; j < m; j++) {
                available[j] += request[j];
                allocation[processId][j] -= request[j];
                need[processId][j] += request[j];
            }
            return false;
        }
    }
}

class PageReplacement {
    public static int fifo(int[] pages, int frameSize) {
        Queue<Integer> queue = new LinkedList<>();
        Set<Integer> set = new HashSet<>();
        int pageFaults = 0;

        for (int page : pages) {
            if (!set.contains(page)) {
                pageFaults++;
                if (queue.size() == frameSize) {
                    int removed = queue.poll();
                    set.remove(removed);
                }
                queue.add(page);
                set.add(page);
            }
        }
        return pageFaults;
    }

    public static int optimal(int[] pages, int frameSize) {
        List<Integer> frames = new ArrayList<>();
        int pageFaults = 0;

        for (int i = 0; i < pages.length; i++) {
            if (!frames.contains(pages[i])) {
                pageFaults++;
                if (frames.size() < frameSize) {
                    frames.add(pages[i]);
                } else {
                    int farthest = -1, indexToReplace = -1;
                    for (int j = 0; j < frames.size(); j++) {
```

```
                        int frame = frames.get(j);
                        int nextUse = Integer.MAX_VALUE;
                        for (int k = i + 1; k < pages.length; k++) {
                            if (pages[k] == frame) {
                                nextUse = k;
                                break;
                            }
                        }
                        if (nextUse > farthest) {
                            farthest = nextUse;
                            indexToReplace = j;
                        }
                    }
                    frames.set(indexToReplace, pages[i]);
                }
            }
        }
        return pageFaults;
    }

    public static int lru(int[] pages, int frameSize) {
        LinkedHashMap<Integer, Integer> map = new LinkedHashMap<>(frameSize,
0.75f, true);
        int pageFaults = 0;

        for (int page : pages) {
            if (!map.containsKey(page)) {
                pageFaults++;
                if (map.size() == frameSize) {
                    int leastRecent = map.keySet().iterator().next();
                    map.remove(leastRecent);
                }
            }
            map.put(page, page);
        }
        return pageFaults;
    }
}

public class CompleteOSSimulator {
    private static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        while (true) {
            System.out.println("\n===== OS Simulator Menu =====");
```

```
            System.out.println("1. CPU Scheduling Algorithms");
            System.out.println("2. Banker's Algorithm");
            System.out.println("3. Page Replacement Algorithms");
            System.out.println("4. Exit");
            System.out.print("Choose option: ");

            String choice = scanner.nextLine();

            switch(choice) {
                case "1":
                    cpuSchedulingMenu();
                    break;
                case "2":
                    bankerAlgorithmMenu();
                    break;
                case "3":
                    pageReplacementMenu();
                    break;
                case "4":
                    System.out.println("Thank you for using OS Simulator!");
                    return;
                default:
                    System.out.println("Invalid choice, please try again.");
            }
        }
    }

    private static void cpuSchedulingMenu() {
        System.out.println("\n----- CPU Scheduling Algorithms -----");

        int n = getPositiveInteger("Enter number of processes: ");

        System.out.print("Do you have arrival times? (yes/no): ");
        String hasArrival = scanner.nextLine().toLowerCase();

        int[] arrivalTimes = new int[n];
        if (hasArrival.equals("yes") || hasArrival.equals("y")) {
            arrivalTimes = getIntegerArray("Enter arrival times (space
separated): ", n);
        } else {
            Arrays.fill(arrivalTimes, 0);
        }

        int[] burstTimes = getIntegerArray("Enter burst times (space separated):
", n);
```

```
        List<Process> processes = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            processes.add(new Process(i + 1, arrivalTimes[i], burstTimes[i]));
        }

        while (true) {
            System.out.println("\n--- Choose Scheduling Algorithm ---");
            System.out.println("1. FCFS (First-Come First-Served)");
            System.out.println("2. Round Robin (RR)");
            System.out.println("3. SJF (Non-preemptive)");
            System.out.println("4. SJF (Preemptive)");
            System.out.println("5. Back to Main Menu");

            String choice = scanner.nextLine();

            switch(choice) {
                case "1":
                    fcfs(copyProcesses(processes));
                    break;
                case "2":
                    int quantum = getPositiveInteger("Enter quantum size: ");
                    roundRobin(copyProcesses(processes), quantum);
                    break;
                case "3":
                    sjfNonPreemptive(copyProcesses(processes));
                    break;
                case "4":
                    sjfPreemptive(copyProcesses(processes));
                    break;
                case "5":
                    return;
                default:
                    System.out.println("Invalid choice!");
            }
        }
    }

    private static void bankerAlgorithmMenu() {
        System.out.println("\n----- Banker's Algorithm -----");

        int n = getPositiveInteger("Enter number of processes: ");
        int m = getPositiveInteger("Enter number of resource types: ");
```

```
        int[] available = getIntegerArray("Enter available resources (space
separated): ", m);

        System.out.println("Enter Allocation Matrix (row by row):");
        int[][] allocation = new int[n][m];
        for (int i = 0; i < n; i++) {
            System.out.print("Row " + (i+1) + ": ");
            allocation[i] = getIntegerArray("", m);
        }

        System.out.println("Enter Max Matrix (row by row):");
        int[][] max = new int[n][m];
        for (int i = 0; i < n; i++) {
            System.out.print("Row " + (i+1) + ": ");
            max[i] = getIntegerArray("", m);
        }

        BankerAlgorithm banker = new BankerAlgorithm(n, m, max, allocation,
available);

        System.out.println("\nNeed Matrix:");
        for (int i = 0; i < n; i++) {
            System.out.print("P" + i + ": ");
            for (int j = 0; j < m; j++) {
                System.out.print(banker.need[i][j] + " ");
            }
            System.out.println();
        }

        if (banker.isSafe()) {
            System.out.println("\nSystem is in SAFE state!");
            System.out.print("Safe Sequence: ");
            for (int i = 0; i < n; i++) {
                System.out.print("P" + banker.safeSequence[i] + " ");
            }
            System.out.println();
        } else {
            System.out.println("\nSystem is in UNSAFE state!");
        }

        System.out.print("\nDo you want to request additional resources?
(yes/no): ");
        String requestChoice = scanner.nextLine().toLowerCase();

        if (requestChoice.equals("yes") || requestChoice.equals("y")) {
```

```
            int processId = getPositiveInteger("Enter process ID: ");
            int[] request = getIntegerArray("Enter request (space separated): ",
m);

            if (banker.requestResources(processId, request)) {
                System.out.println("Request GRANTED! System remains in safe
state.");
            } else {
                System.out.println("Request DENIED! May lead to unsafe state.");
            }
        }
    }

    private static void pageReplacementMenu() {
        System.out.println("\n----- Page Replacement Algorithms -----");

        int frameSize = getPositiveInteger("Enter frame size: ");

        System.out.print("Enter reference string (space separated): ");
        String[] tokens = scanner.nextLine().split("\\s+");
        int[] pages = new int[tokens.length];
        for (int i = 0; i < tokens.length; i++) {
            pages[i] = Integer.parseInt(tokens[i]);
        }

        int fifoFaults = PageReplacement.fifo(pages, frameSize);
        int optimalFaults = PageReplacement.optimal(pages, frameSize);
        int lruFaults = PageReplacement.lru(pages, frameSize);

        int totalReferences = pages.length;

        System.out.println("\n----- Page Replacement Results -----");
        System.out.println("Total References: " + totalReferences);
        System.out.println("Frame Size: " + frameSize);
        System.out.println("Reference String: " + Arrays.toString(pages));
        System.out.println();

        System.out.println("+-----------------+-------------+------------+-----
---------+");
        System.out.println("| Algorithm       | Page Faults  | Hit Ratio   | Miss
Ratio   |");
        System.out.println("+-----------------+-------------+------------+-----
---------+");

        printRow("FIFO", fifoFaults, totalReferences);
```

```
        printRow("Optimal", optimalFaults, totalReferences);
        printRow("LRU", lruFaults, totalReferences);

        System.out.println("+----------------+-------------+------------+-----
---------+");
    }

    private static void printRow(String algorithm, int faults, int total) {
        double hitRatio = (double)(total - faults) / total * 100;
        double missRatio = (double)faults / total * 100;

        System.out.printf("| %-15s | %-12d | %-11.2f%% | %-12.2f%% |\n",
                          algorithm, faults, hitRatio, missRatio);
    }

    private static int getPositiveInteger(String message) {
        while (true) {
            try {
                System.out.print(message);
                int value = Integer.parseInt(scanner.nextLine());
                if (value > 0) return value;
                System.out.println("Value must be positive!");
            } catch (NumberFormatException e) {
                System.out.println("Invalid input! Please enter an integer.");
            }
        }
    }

    private static int[] getIntegerArray(String message, int expectedSize) {
        while (true) {
            try {
                System.out.print(message);
                String[] tokens = scanner.nextLine().split("\\s+");
                if (tokens.length != expectedSize) {
                    System.out.println("Please enter exactly " + expectedSize + "
numbers!");
                    continue;
                }
                int[] result = new int[expectedSize];
                for (int i = 0; i < expectedSize; i++) {
                    result[i] = Integer.parseInt(tokens[i]);
                }
                return result;
            } catch (NumberFormatException e) {
                System.out.println("Invalid input! Please enter numbers only.");
```

```
            }
        }
    }

    static List<Process> copyProcesses(List<Process> original) {
        List<Process> copy = new ArrayList<>();
        for (Process p : original) {
            copy.add(new Process(p.id, p.arrivalTime, p.burstTime));
        }
        return copy;
    }

    static void fcfs(List<Process> processes) {
        processes.sort(Comparator.comparingInt(p -> p.arrivalTime));
        int time = 0;
        for (Process p : processes) {
            if (time < p.arrivalTime) time = p.arrivalTime;
            p.executionLog.add(new int[]{time, time + p.burstTime});
            p.waitingTime = time - p.arrivalTime;
            time += p.burstTime;
            p.turnaroundTime = p.waitingTime + p.burstTime;
        }
        printResult("FCFS", processes);
    }

    static void roundRobin(List<Process> processes, int quantum) {
        Queue<Process> queue = new LinkedList<>();
        int time = 0, completed = 0;
        List<Process> arrived = new ArrayList<>(processes);
        arrived.sort(Comparator.comparingInt(p -> p.arrivalTime));
        Set<Integer> inQueue = new HashSet<>();

        while (completed < processes.size()) {
            for (Process p : arrived) {
                if (p.arrivalTime <= time && !inQueue.contains(p.id) &&
p.remainingTime > 0) {
                    queue.add(p);
                    inQueue.add(p.id);
                }
            }

            if (queue.isEmpty()) {
                time++;
                continue;
            }
```

```
            Process current = queue.poll();
            int executeTime = Math.min(quantum, current.remainingTime);
            if (current.startTime == -1) current.startTime = time;
            current.executionLog.add(new int[]{time, time + executeTime});
            time += executeTime;
            current.remainingTime -= executeTime;

            for (Process p : arrived) {
                if (p.arrivalTime <= time && !inQueue.contains(p.id) &&
p.remainingTime > 0) {
                    queue.add(p);
                    inQueue.add(p.id);
                }
            }

            if (current.remainingTime > 0) {
                queue.add(current);
            } else {
                current.turnaroundTime = time - current.arrivalTime;
                current.waitingTime = current.turnaroundTime - current.burstTime;
                completed++;
            }
        }
        processes.sort(Comparator.comparingInt(p -> p.id));
        printResult("Round Robin", processes);
    }

    static void sjfNonPreemptive(List<Process> processes) {
        List<Process> result = new ArrayList<>();
        int time = 0;
        while (!processes.isEmpty()) {
            List<Process> available = new ArrayList<>();
            for (Process p : processes) {
                if (p.arrivalTime <= time) available.add(p);
            }
            if (available.isEmpty()) {
                time++;
                continue;
            }
            available.sort(Comparator.comparingInt(p -> p.burstTime));
            Process current = available.get(0);
            processes.remove(current);
            current.executionLog.add(new int[]{time, time + current.burstTime});
            current.waitingTime = time - current.arrivalTime;
```

```
            time += current.burstTime;
            current.turnaroundTime = current.waitingTime + current.burstTime;
            result.add(current);
        }
        result.sort(Comparator.comparingInt(p -> p.id));
        printResult("SJF (Non-preemptive)", result);
    }

    static void sjfPreemptive(List<Process> processes) {
        int time = 0, completed = 0;
        Process current = null;
        while (completed < processes.size()) {
            Process shortest = null;
            for (Process p : processes) {
                if (p.arrivalTime <= time && p.remainingTime > 0) {
                    if (shortest == null || p.remainingTime <
shortest.remainingTime) {
                        shortest = p;
                    }
                }
            }
            if (shortest == null) {
                time++;
                continue;
            }
            if (shortest.startTime == -1) shortest.startTime = time;
            if (current != shortest) {
                current = shortest;
                current.executionLog.add(new int[]{time, time + 1});
            } else {
                int[] last =
current.executionLog.remove(current.executionLog.size() - 1);
                current.executionLog.add(new int[]{last[0], time + 1});
            }
            shortest.remainingTime--;
            time++;
            if (shortest.remainingTime == 0) {
                shortest.turnaroundTime = time - shortest.arrivalTime;
                shortest.waitingTime = shortest.turnaroundTime -
shortest.burstTime;
                completed++;
            }
        }
        processes.sort(Comparator.comparingInt(p -> p.id));
        printResult("SJF (Preemptive)", processes);
```

```
    }

    static void printResult(String title, List<Process> processes) {
        System.out.println("\n--- " + title + " Algorithm ---\n");
        System.out.println("Process ID\tArrival Time\tBurst Time\tWaiting
Time\tTurnaround Time");
        int totalWT = 0, totalTAT = 0;
        for (Process p : processes) {
            System.out.printf("%d\t\t%d\t\t%d\t\t%d ms\t\t%d ms\n",
                p.id, p.arrivalTime, p.burstTime, p.waitingTime,
p.turnaroundTime);
            totalWT += p.waitingTime;
            totalTAT += p.turnaroundTime;
        }
        System.out.printf("Average Waiting Time: %.2f ms\n", (double) totalWT /
processes.size());
        System.out.printf("Average Turnaround Time: %.2f ms\n", (double) totalTAT
/ processes.size());

        System.out.println("\nGantt Chart:");
        for (Process p : processes) {
            for (int[] slot : p.executionLog) {
                System.out.print("| P" + p.id + " ");
            }
        }
        System.out.println("|");

        List<int[]> timeline = new ArrayList<>();
        for (Process p : processes) {
            timeline.addAll(p.executionLog);
        }
        timeline.sort(Comparator.comparingInt(a -> a[0]));

        System.out.print("Time: ");
        for (int[] slot : timeline) {
            System.out.print(slot[0] + " ");
        }
        if (!timeline.isEmpty()) {
            System.out.println(timeline.get(timeline.size() - 1)[1]);
        }
    }
}
```

Executive Summary

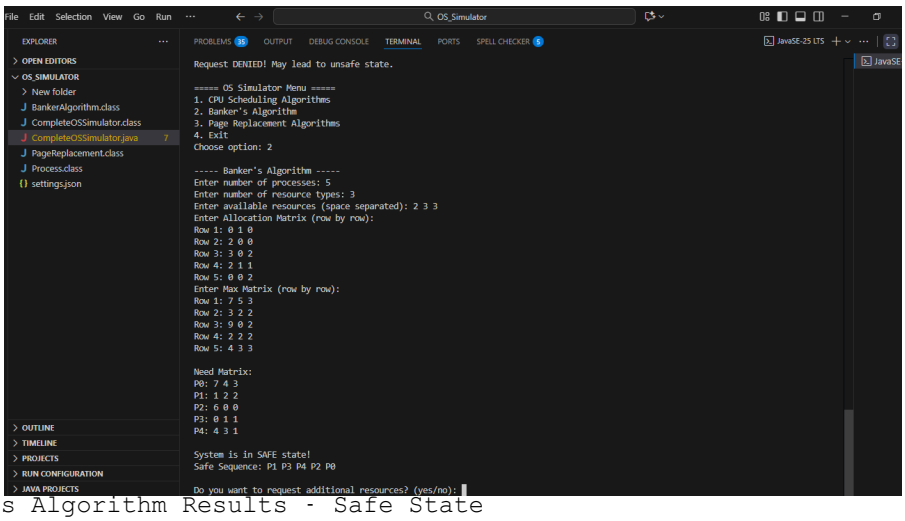
A comprehensive operating system simulator has been developed featuring three main modules:

1. **CPU Scheduling Algorithms** - FCFS, Round Robin, SJF (Non-preemptive and Preemptive)
2. **Banker's Algorithm** - For safety checks and resource request handling
3. **Page Replacement Algorithms** - FIFO, Optimal, LRU

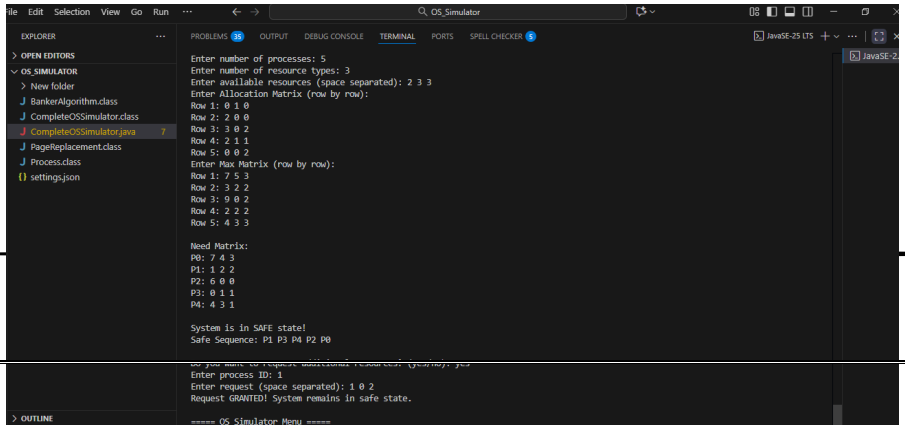
1. Banker's Algorithm

Fully implemented with the following features:

- Automatic Need Matrix calculation
- Safety state verification
- Safe sequence identification
- Additional resource request processing



Banker's Algorithm Results - Safe State

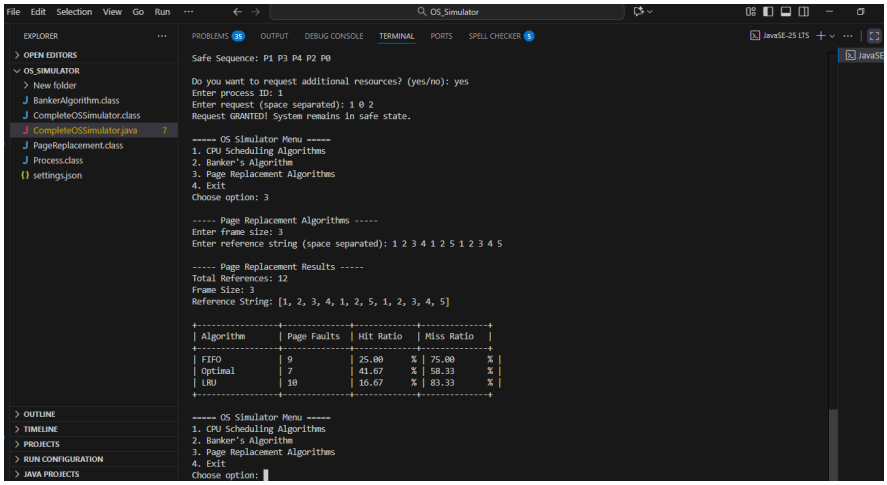


Additional Resource Request Handling

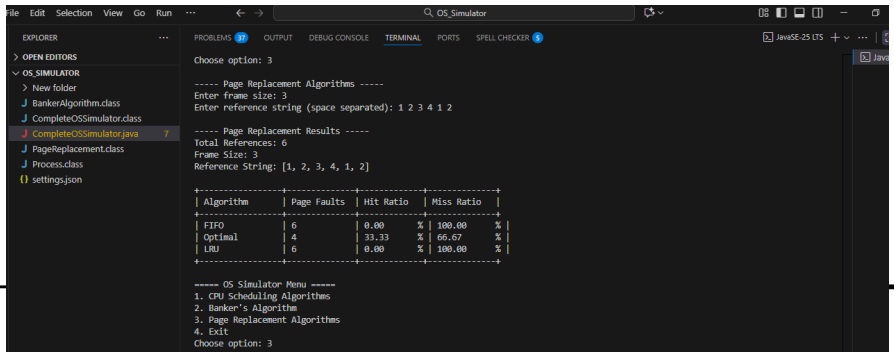
2. Page Replacement Algorithms

Three algorithms implemented with performance comparison:

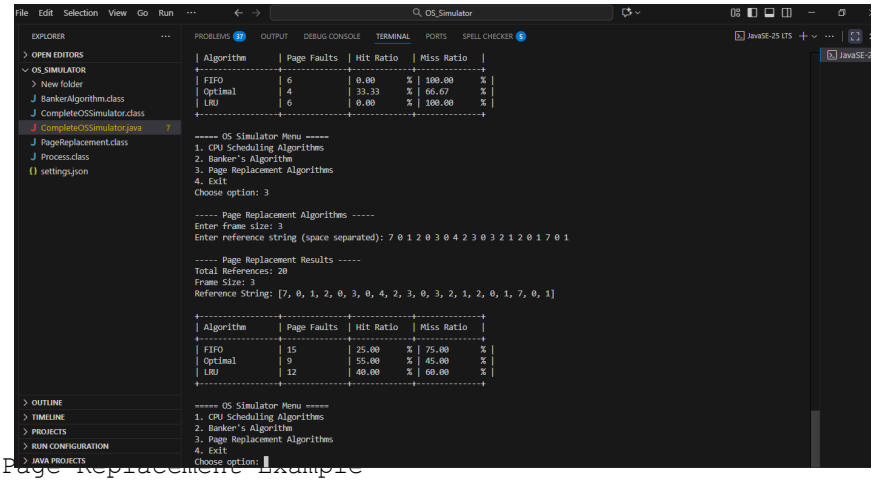
1. **FIFO** (First-In-First-Out)
2. **Optimal**
3. **LRU** (Least Recently Used)



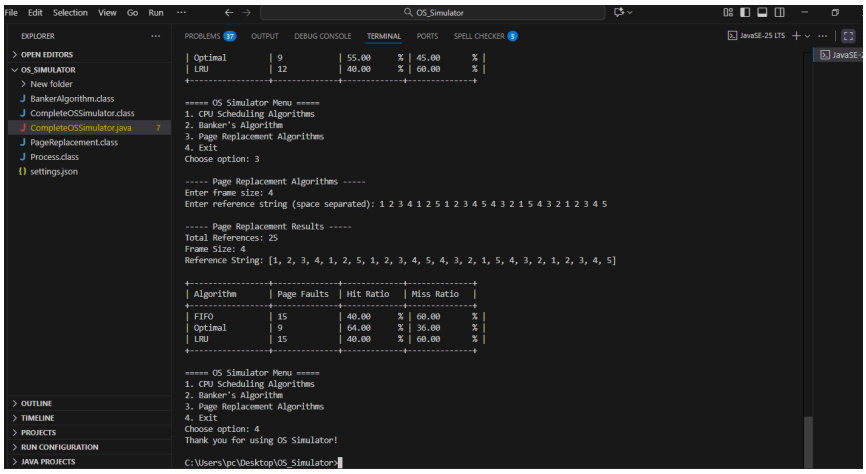
Page Replacement Algorithms Comparison - Simple Example



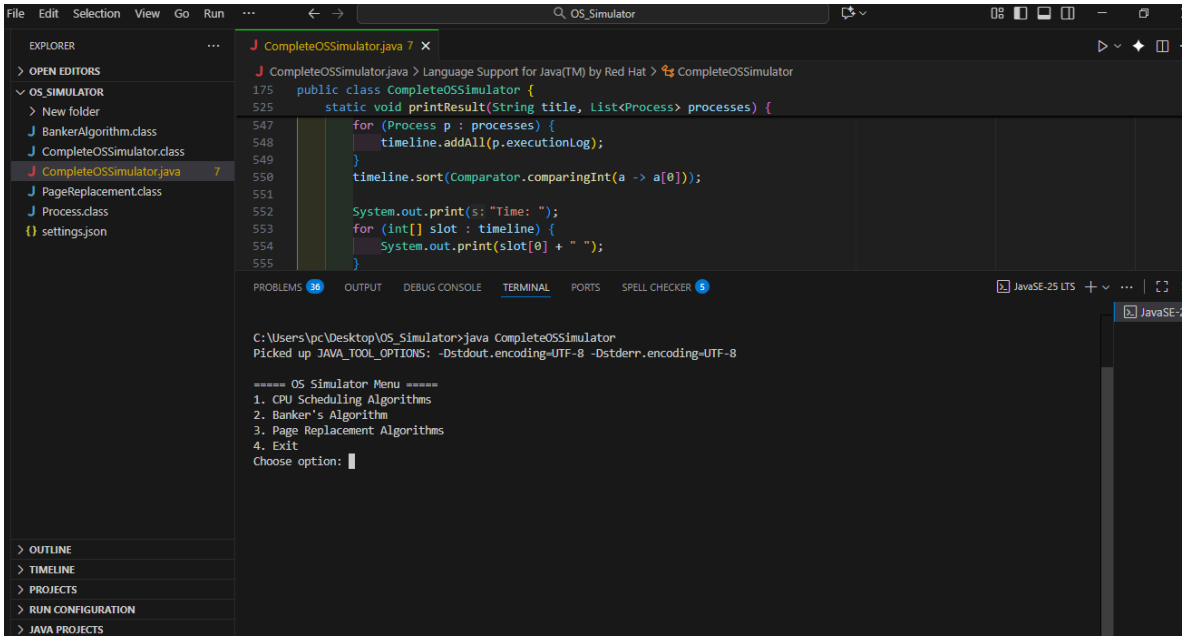
Medium Complexity Page Replacement Example



Complex Page Replacement Example



A unified main interface developed providing access to all algorithms:

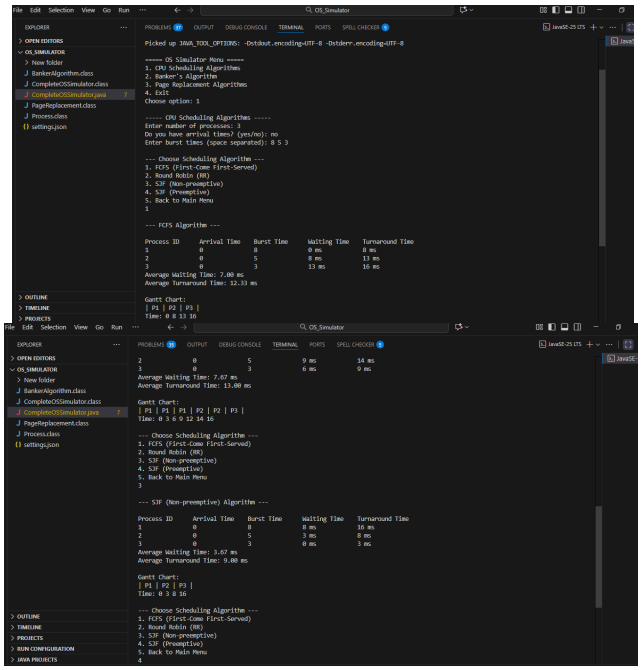


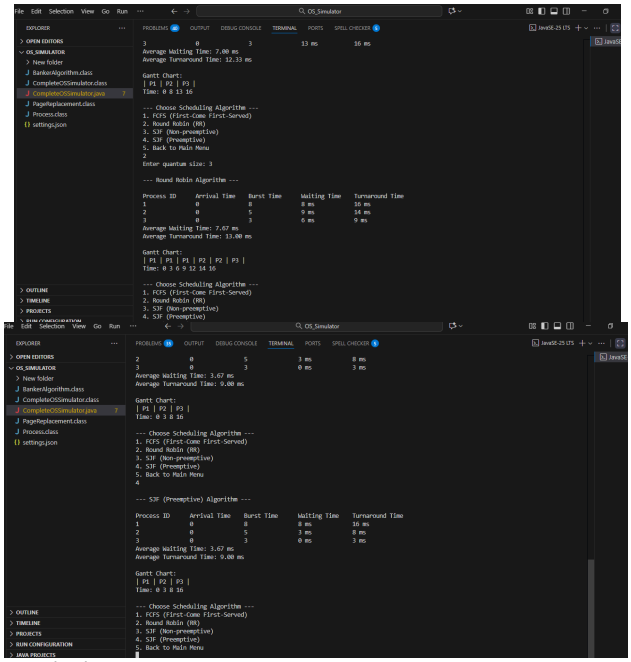
Simulator Main Menu

Results and Evaluation

CPU Scheduling

Different CPU Scheduling Algorithm Results





- ***Banker's Algorithm**: Successfully identified safe and unsafe states accurately*

- ***Page Replacement**: Optimal showed best performance followed by LRU then FIFO*

Conclusion

The implementation of the CPU scheduling algorithms demonstrated the differences in process scheduling and performance. We observed that FCFS and SJF (Non-preemptive) had similar results, while Round Robin showed higher average waiting times due to its time-sharing nature. SJF (Preemptive) was found to be the most efficient algorithm, minimizing both waiting time and turnaround time. Each algorithm has its own advantages and limitations, and their choice depends on the system's needs. Moreover, the simulator incorporates effective error handling through try-catch blocks, especially for user inputs such as number of processes, arrival times, burst times, and quantum values. This helps maintain program stability and guides users toward correct input formats.

Successfully developed a comprehensive simulator that meets all project requirements. The program represents a valuable educational tool for understanding various operating system mechanisms.